When Ruby executes obj.tickle in this code example, it does the usual trick of looking in the class of obj for a method called tickle. For extend to work, it has to add the instance methods in the Humor module into the superclass chain for the class of obj. So, just as with singleton method definitions, Ruby creates a singleton class for obj and then includes the module Humor in that class. To prove that this is all that happens, here's the C implementation of extend in the current Ruby 3.2 interpreter:

```
void
rb_extend_object(VALUE obj, VALUE module)
{
    rb_include_module(rb_singleton_class(obj), module);
}
```

There is an interesting trick with extend. If you use it within a class definition, the module's methods become class methods. This is because calling extend is equivalent to self.extend, so the methods are added to self, which in a class definition is the class itself.

Here's an example of adding a module's methods at the class level:

```
module Humor
  def tickle
    "#{self} says hee, hee!"
  end
end

class Grouchy
  extend Humor
end

Grouchy.tickle # => "Grouchy says hee, hee!"
```

Later, in Class Methods and Modules, on page 392, we'll see how to use extend to add macro-style methods to a class.

Using Refinements

We previously looked at applying a change to a builtin class by defining the altered version of a method in a module, and then prepending that module in the class. When we subsequently call the method on instances of the class, it finds the version in the module first. We've also seen that Ruby allows you to re-open an existing classes or module and add new methods to it. When we subsequently use instances of the class, the new methods exist along side the ones that were previously defined.

These techniques are time-honored, and frameworks such as Ruby on Rails rely on them. But they come with a price. Any changes we make with prepend and monkey patches are global to our running application. They apply not just to the code we wrote for our application, but also to the code in all the libraries and gems we use while the application is running, too.

It is possible that a change that made our code easier to write breaks someone else's library code that we rely on. This is clearly a problem in theory. Does it happen in practice? Actually, surprisingly rarely. But you can never be sure that things will quite work as you expect. Even if you don't override these classes yourself, it is possible you're using two separate libraries whose patches to third-party classes clash.